Restaurante Rei do Filet Ltda.: Sistema de Comandas

GRUPO 4:

Gabriela Garcia
Gustavo Henrique de Moraes
Lorenzzo Paulelli
Márcia Raissa
Sofia Otsuki

Disciplina: Engenharia de Software III - IES300
Empresa: Restaurante Rei do Filet
Sistema: Sistema de Atendimento e Comandas

Data: 12 de setembro de 2026

Índice

Índice **2**
Introdução **3**
Capítulo I - Fundamentação do Estilo Arquitetural na Plataforma Escolhida **3**
Capítulo II - Preparação do Ambiente e Estrutura do Projeto **4**
2.1 Estrutura de pastas do projeto 4
2.2 Configuração do ambiente e das dependências 4
Capítulo III - Modelagem de Dados e Camada de Persistência **5**
Capítulo IV - Implementação das Camadas de Serviço e de API REST **5**
4.1 Camada de serviços (regras de negócio) 5
4.2 Camada de rotas (endpoints REST) 6
Capítulo V - Segurança e Testes Automatizados **6**
5.1 Autenticação e permissões 6
5.2 Testes automatizados 6
Capítulo VI - Implantação em Nuvem (Deploy) **6**
6.1 Banco de dados (Neon.tech) 7
6.2 Backend (Render) 7
6.3 Frontend (Vercel) e integração com a API 7
6.4 Monitoramento e evolução contínua 8
Considerações Finais **8**
Referências **8**

Introdução

Este documento complementa o Projeto de Arquitetura de Software do Sistema de Atendimento e Comandas do Restaurante Rei do Filet, detalhando como o estilo arquitetural escolhido — Cliente-Servidor em Camadas Lógicas (Layered Architecture), orientado a serviços RESTful — é efetivamente implementado na plataforma tecnológica definida pela equipe: back-end em Python com FastAPI, front-end em React, banco de dados PostgreSQL 16 hospedado no Neon.tech, com implantação do back-end no Render e do front-end na Vercel.

O objetivo é apresentar, de forma pesquisada e fundamentada em práticas atuais de mercado, um passo a passo prático de construção da aplicação, cobrindo desde a organização das pastas do projeto até a publicação da solução em ambiente de nuvem, de modo que qualquer integrante da equipe consiga reproduzir e evoluir o sistema de forma padronizada.

Capítulo I - Fundamentação do Estilo Arquitetural na Plataforma Escolhida

A Layered Architecture organiza o sistema em camadas com responsabilidades únicas, onde cada camada só se comunica com a camada imediatamente adjacente. Na prática de mercado consolidada para aplicações FastAPI em produção, essa organização costuma ser expressa em quatro a cinco camadas de código, complementadas por uma camada de infraestrutura transversal:

- Camada de Rotas/Controladores (Routers): recebe requisições HTTP, aplica validação de entrada via Pydantic e delega a execução para a camada de serviços — nunca contém regra de negócio ou acesso direto ao banco.
- Camada de Esquemas (Schemas/DTOs): define os contratos de entrada e saída da API (Pydantic models), desacoplando o formato exposto externamente do modelo interno de dados.
- Camada de Serviços (Services): concentra as regras de negócio do restaurante — abertura/fechamento de comanda, transições de status de mesa e reserva, cálculo de totais — orquestrando chamadas à camada de persistência.
- Camada de Repositórios/Persistência (Repositories): isola todo o acesso a dados (consultas SQLAlchemy), permitindo trocar detalhes de armazenamento sem afetar as regras de negócio.
- Camada de Modelos (Models): mapeia as entidades do domínio (Mesa, Comanda, Pedido, Produto etc.) às tabelas do PostgreSQL.
- Núcleo transversal (Core): configuração de ambiente, segurança/JWT, tratamento de exceções e logging, utilizados por todas as camadas acima.

Essa separação estrita — a API não acessa o banco diretamente, e o serviço não conhece detalhes de HTTP — é o que garante o desacoplamento definido no Projeto de Arquitetura,

permitindo que o time evolua o front-end React, o backend FastAPI e o banco PostgreSQL de forma independente.

Capítulo II - Preparação do Ambiente e Estrutura do Projeto

2.1 Estrutura de pastas do projeto

Antes de escrever código, a equipe define a árvore de diretórios do backend, refletindo diretamente as camadas do estilo arquitetural. A estrutura abaixo segue o padrão adotado atualmente em projetos FastAPI de produção:

```
app/
├── main.py                 # instancia o FastAPI e registra os routers
├── core/
│   ├── config.py           # variáveis de ambiente (Pydantic Settings)
│   ├── security.py         # geração/validação de tokens JWT
│   └── exceptions.py       # exceções e handlers customizados
├── db/
│   ├── session.py          # engine e sessão assíncrona SQLAlchemy
│   └── base.py             # Base declarativa dos modelos
├── models/                 # entidades (Mesa, Comanda, Pedido, Produto...)
├── schemas/                # contratos Pydantic de entrada/saída
├── repositories/           # consultas e comandos SQL por entidade
├── services/               # regras de negócio (Comanda, Reserva, Pagamento)
├── routers/                # endpoints REST agrupados por recurso
└── tests/              # testes unitários e de integração
```

Estrutura de referência para organização por camadas em FastAPI (routers → services → repositories → models).

2.2 Configuração do ambiente e das dependências

Cria-se um ambiente virtual Python e um arquivo de dependências mínimo para a API assíncrona:

```
python -m venv .venv && source .venv/bin/activate
 pip install fastapi uvicorn[standard] sqlalchemy[asyncio] asyncpg
 pip install pydantic-settings python-jose passlib[bcrypt]
 pip freeze > requirements.txt
```

As credenciais de acesso ao banco (fornecidas pelo Neon.tech) e demais segredos são mantidos fora do código-fonte, em variáveis de ambiente lidas por um objeto de configuração central (core/config.py), evitando que a string de conexão seja versionada no GitHub.

Capítulo III - Modelagem de Dados e Camada de Persistência

As nove entidades definidas no Projeto de Banco de Dados (Funcionario, Mesa, Reserva, Categoria, Produto, Comanda, Pedido, Item_Pedido e Pagamento) são traduzidas em classes SQLAlchemy dentro de models/, uma por arquivo. Em seguida, cada entidade recebe um repositório correspondente em repositories/, responsável exclusivamente por operações de leitura e escrita:

```
class ComandaRepository:
        def __init__(self, session: AsyncSession):
        self.session = session

        async def obter_ativa_por_mesa(self, id_mesa: int) -> Comanda | None:
        stmt = select(Comanda).where(
        Comanda.id_mesa == id_mesa,
        Comanda.status_comanda == 'ABERTA'
        )
        result = await self.session.execute(stmt)
        return result.scalar_one_or_none()
```

A criação das tabelas em si é feita por meio de migrações versionadas (Alembic), aplicadas contra o banco PostgreSQL 16 provisionado no Neon.tech, garantindo que o esquema físico evolua de forma controlada e rastreável junto com o código.

Capítulo IV - Implementação das Camadas de Serviço e de API REST

4.1 Camada de serviços (regras de negócio)

Cada serviço recebe um ou mais repositórios por injeção de dependência e implementa as regras descritas nas Regras de Negócio e nos Casos de Uso da especificação, como o ciclo de vida da reserva (Confirmada → Concluída/Cancelada) e a abertura automática de comanda no check-in:

```
class ReservaService:
        def __init__(self, reservas: ReservaRepository, mesas: MesaRepository):
        self.reservas = reservas
        self.mesas = mesas

        async def checkin(self, id_reserva: int) -> Comanda:
        reserva = await self.reservas.obter(id_reserva)
        reserva.status_reserva = 'CONCLUIDA'
        await self.mesas.atualizar_status(reserva.id_mesa, 'OCUPADA')
        return await self.comandas.abrir(reserva.id_mesa)
```

É nessa camada que residem validações como o congelamento do preço unitário em Item_Pedido e o bloqueio de alteração em comandas já finalizadas (RN03), mantendo a lógica de domínio isolada de qualquer detalhe de banco de dados ou de protocolo HTTP.

4.2 Camada de rotas (endpoints REST)

Os routers traduzem cada caso de uso em um endpoint HTTP, usando Pydantic para validar entrada e formatar a resposta, e delegando toda a lógica ao serviço correspondente:

```
router = APIRouter(prefix="/comandas", tags=["Comandas"])

@router.post("/{id_mesa}/itens", response_model=ItemPedidoOut)
async def registrar_item(
        id_mesa: int,
        item: ItemPedidoCreate,
        service: ComandaService = Depends(get_comanda_service),
):
        return await service.registrar_item(id_mesa, item)
```

A documentação interativa (Swagger/OpenAPI) é gerada automaticamente pelo FastAPI a partir dessas assinaturas, sem esforço adicional da equipe, conforme já apontado na comparação de trade-offs do documento de arquitetura.

Capítulo V - Segurança e Testes Automatizados

5.1 Autenticação e permissões

A camada core/security.py implementa emissão e validação de tokens JWT para os perfis Funcionário, Administrador e Contabilidade. Um dependency de FastAPI intercepta as rotas protegidas e verifica o perfil antes de acionar o serviço, atendendo aos requisitos de gestão de usuários e permissões e à regra de que apenas administradores alterem comandas finalizadas.

5.2 Testes automatizados

Cada camada é testada isoladamente: testes unitários para services (com repositórios substituídos por dublês/mocks) e testes de integração para routers, executados contra um banco de testes. Isso garante que a separação de camadas realmente permita testar regra de negócio sem subir um banco de dados real.

```
pytest tests/unit          # regras de negócio isoladas
pytest tests/integration   # endpoints + banco de testes
```

Capítulo VI - Implantação em Nuvem (Deploy)

6.1 Banco de dados (Neon.tech)

1. Criar uma conta no Neon.tech e um novo projeto PostgreSQL 16.
2. Copiar a connection string fornecida (formato postgres://usuario:senha@host/neondb?sslmode=require).
3. Definir essa string como variável de ambiente DATABASE_URL, nunca diretamente no código.
4. Executar as migrações Alembic contra o banco na nuvem para criar as tabelas definidas no modelo físico.

Por ser serverless, o Neon.tech escala e hiberna automaticamente conforme o uso, o que é adequado ao perfil de tráfego de um restaurante de médio porte, sem custo de infraestrutura ociosa fora do horário de funcionamento.

6.2 Backend (Render)

1. Subir o repositório do backend para o GitHub, incluindo requirements.txt.
2. No painel do Render, criar um novo Web Service apontando para o repositório.
3. Definir o comando de inicialização, sempre lendo a porta pela variável de ambiente do Render:

```
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

4. Cadastrar a variável DATABASE_URL (apontando para o Neon.tech) e demais segredos no painel do serviço.
5. Configurar um monitor externo (UptimeRobot) fazendo ping periódico ao serviço, evitando a hibernação do plano gratuito do Render mencionada como risco no estudo de viabilidade.

6.3 Frontend (Vercel) e integração com a API

1. Gerar o build de produção do React (ex.: Vite) e conectar o repositório do front-end à Vercel.
2. Definir a URL pública do backend do Render como variável de ambiente de build (ex.: VITE_API_URL), consumida pelo cliente HTTP do React.
3. No backend FastAPI, habilitar CORS liberando explicitamente o domínio gerado pela Vercel:

```
app.add_middleware(
        CORSMiddleware,
        allow_origins=["https://reidofilet.vercel.app"],
        allow_methods=["*"],
        allow_headers=["*"],
)
```

4. Validar, em ambiente de homologação, o fluxo completo descrito nos casos de uso: mapa de mesas, abertura de comanda, envio à cozinha, pré-fechamento e pagamento.

6.4 Monitoramento e evolução contínua

Com os três serviços publicados (Vercel, Render e Neon.tech) e o repositório versionado no GitHub, o time passa a acompanhar logs e métricas de cada camada de forma independente, o que confirma na prática o benefício central do estilo arquitetural escolhido: qualquer camada — interface, regra de negócio ou armazenamento — pode ser depurada, escalada ou substituída sem exigir mudanças nas demais.

Considerações Finais

O passo a passo apresentado demonstra que o estilo Cliente-Servidor em Camadas não é apenas uma decisão conceitual registrada no documento de arquitetura, mas um conjunto de práticas concretas de organização de pastas, injeção de dependências e pipeline de implantação. Ao seguir routers → services → repositories → models no backend, e ao publicar front-end, backend e banco de dados como serviços independentes (Vercel, Render e Neon.tech), a equipe garante o desacoplamento, a escalabilidade e a facilidade de manutenção que motivaram a escolha dessa arquitetura para o Sistema de Atendimento e Comandas do Restaurante Rei do Filet.

Referências

DOBHAL, Abhinav. Building production-ready FastAPI applications with service layer architecture in 2025. Medium, 2025. Disponível em: <https://medium.com/@abhinav.dobhal/building-production-ready-fastapi-applications-with-service-layer-architecture-in-2025-f3af8a6ac563>. Acesso em: 12 set. 2026.

DEV COMMUNITY. Deploying a FastAPI app on Render: a step-by-step guide. 2026. Disponível em: <https://dev.to/ganesh_bora_12e6afdf2c7f0/deploying-a-fastapi-app-on-render-a-step-by-step-guide-4nno>. Acesso em: 12 set. 2026.

DEV COMMUNITY. Vercel + Render hybrid deployment: why I split my stack (and how). 2026. Disponível em: <https://dev.to/uaslimcreate/vercel-render-hybrid-deployment-why-i-split-my-stack-and-how-34n2>. Acesso em: 12 set. 2026.

FASTAPI. Documentação oficial e arquitetura de APIs modernas em Python. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 12 set. 2026.

FREECODECAMP. How to deploy your FastAPI + PostgreSQL app on Render: a beginner's guide. 2025. Disponível em: <https://www.freecodecamp.org/news/deploy-fastapi-postgresql-app-on-render/>. Acesso em: 12 set. 2026.

GRUPO 4. Projeto de Arquitetura e Modelagem de Banco de Dados — Sistema de Atendimento e Comandas. Engenharia de Software III (IES300), 2026.

NEON. Building an async product management API with FastAPI, Pydantic, and PostgreSQL. Neon Guides. Disponível em: <https://neon.com/guides/fastapi-async>. Acesso em: 12 set. 2026.

VERCEL. Deploy a FastAPI app on Vercel. Disponível em: <https://vercel.com/docs/frameworks/backend/fastapi>. Acesso em: 12 set. 2026.

ZESTMINDS. FastAPI project structure: production guide 2026. Disponível em: <https://www.zestminds.com/blog/fastapi-project-structure/>. Acesso em: 12 set. 2026.